

Non-Graded Project: Build Complete Customer Support Assistant with LangChain

Description

Transform GlobalSupport Solutions' legacy support infrastructure into a production-ready LangChain application by applying all course skills: building modular components, applying systematic refactoring methodology, and implementing production patterns for scalability and reliability.

Learning Objectives

By the end of this project, learners will be able to:

- Construct a complete LangChain application integrating PromptTemplates, LLM abstractions, and OutputParsers (LO1)
- Apply the 5-step refactoring methodology to transform provided legacy code into maintainable architecture (LO2)
- Implement production-ready RAG system with caching, monitoring, and auto-scaling strategies (LO3)



- Synthesize all course learnings into a professional, deployable customer support solution
- Evaluate architectural trade-offs and make informed decisions balancing performance, cost, and reliability



Project Components

Introductory Slide:

Title: Lead Engineer Mission: Transform GlobalSupport Solutions or Lose 30% Market Share

Subtitle: 60 minutes to prove LangChain can rebuild a failing support platform serving 1M+ daily queries

Tools to be used:

- LangChain (Python framework)
- OpenAI GPT-4 or Claude API
- Pinecone or Weaviate (vector database for RAG)
- Redis (caching layer)
- Docker/Kubernetes (containerization and orchestration)
- Prometheus & Grafana (monitoring)
- pytest (testing framework)
- Git for version control

Scenario

GlobalSupport Solutions, a major customer service platform provider, serves 1,000+ enterprise clients including Fortune 500 companies in finance, healthcare, and retail. Following industry disruptions where AI competitors captured 30% market share, the CEO initiated "Project Phoenix"—complete transformation of their legacy support infrastructure. You've been hired as Lead LLM Engineer after recent failures in competitors' AI systems (hallucinations, data leaks, inconsistent responses) made security and reliability non-negotiable.

Your clients demand: 1M+ daily queries across multiple languages, 99.9% uptime, consistent brand-appropriate responses, and seamless integration with existing CRM systems. The current system has 2,000+ lines of monolithic code with hardcoded prompts, no testing, crashes weekly, and takes 6 hours to modify simple responses. Your mission: Build a production-ready customer support assistant demonstrating mastery of all course concepts. You must refactor provided legacy code using systematic methodology, implement modular

LangChain architecture with proper components, integrate RAG for knowledge retrieval, add multi-tier caching for cost optimization, implement comprehensive monitoring, and create deployment documentation. The board reviews your solution Friday—success secures \$10M funding, failure means layoffs.

Instructions for learners

Step 1: Audit and Architecture Design

- – Download the provided legacy support_bot.py codebase (2000+ lines)
- – Conduct systematic audit using the 5-step methodology checklist
- – Document specific problems: hardcoded prompts count, API calls, error handling gaps
- – Create dependency map showing function relationships and coupling points

- 
- Design target LangChain architecture: components, data flow, integration points
 - Map legacy problems to specific LangChain solutions (PromptTemplate, LLM, OutputParser)

Step 2: Core LangChain Components Implementation

- – Extract all prompts into reusable PromptTemplate modules organized by use case
- – Implement LLM abstraction layer with ChatOpenAI/ChatAnthropic supporting provider swapping
- – Create Pydantic OutputParser schemas for intent classification, response generation, sentiment analysis
- – Build modular chain architecture: IntentChain, ResponseChain, FallbackChain
- – Implement proper error handling with logging and graceful degradation

- 
- Move all configuration (API keys, model parameters, prompts) to external YAML/JSON files
 - Organize code structure: prompts/, chains/, parsers/, config/, utils/

Step 3: RAG System Integration

- – Set up vector database (Pinecone/Weaviate) for knowledge base storage
- – Implement document chunking strategy for FAQ and policy documents
- – Build RAG pipeline using LangChain's RetrievalQA with semantic search
- – Configure embeddings model and similarity threshold for retrieval
- – Add source citation requirements for compliance and traceability
- – Test RAG accuracy with sample customer queries

Step 4: Production Patterns - Caching and Monitoring

- – Implement Redis caching layer with intelligent TTL strategy (1-24 hours based on content type)
- – Design cache key structure using query fingerprinting
- – Add cache invalidation hooks for urgent updates
- – Configure Prometheus metrics: latency percentiles (p50, p95, p99), cache hit rate, token consumption
- – Create Grafana dashboard visualizing system health and performance
- – Set up alerting rules for SLA violations and degradation warnings
- – Implement cost tracking showing savings from caching

Step 5: Testing and Deployment Strategy

- – Write comprehensive pytest test suite: unit tests for components, integration tests for chains
- – Implement behavior verification tests comparing refactored vs original output
- – Achieve 80%+ code coverage with edge case testing
- – Create Docker containerization with docker-compose for all services
- – Document Kubernetes deployment with auto-scaling configuration
- – Design strangler fig migration strategy with traffic shifting plan
- – Prepare rollback procedures for immediate reversion if issues arise

Step 6: Professional Documentation and Analysis

- – Create executive summary highlighting key improvements: 70% code reduction, 60% cost savings, 99.9% uptime
- – Document architectural decisions with trade-off analysis (cost vs performance)
- – Compile before/after metrics: lines of code, test coverage, response times, costs
- – Write deployment runbook covering operations, monitoring, troubleshooting
- – Reflect on methodology application and lessons learned (400-500 words)
- – Prepare professional presentation suitable for board review

Deliverable for Submission (AI Grading)

Deliverable 1: Complete Application Architecture

→ **Question Prompt:** What LangChain application architecture did you design to handle GlobalSupport Solutions' 1M+ daily queries, and how do your component choices (intent classification, conversation management, RAG integration, fallback mechanisms) address the critical failures in the legacy system while meeting enterprise requirements for 99.9% reliability?

→ **Response Type:** File Upload

→ **File Types:** Coding files (.py, .json, .yaml, .yml, .md), Compressed archives (.zip)

Deliverable 2: Refactoring Documentation Suite

→ **Question Prompt:** How did you apply the 5-step refactoring methodology to systematically transform the legacy 2,000-line system into modular LangChain architecture, and why does your approach (audit findings, dependency mapping, modularization sequence, testing strategy) minimize production risk while achieving the 70% code reduction target?

→ **Response Type:** File Upload

→ **File Types:** Text (.txt, .pdf, .docx, .md), Image (.png, .jpg for diagrams), Video (.mp4, .mov - max 15 min for walkthrough)

Deliverable 3: Production Deployment Package

→ **Question Prompt:** What production patterns (containerization, caching, monitoring, auto-scaling) did you implement to achieve 99.9% uptime, and how does each pattern specifically address the enterprise deployment challenges that caused competitor AI systems to fail?

→ **Response Type:** File Upload

→ **File Types:** Coding files (.yml, .yaml, .json, .py for configs), Image (.png, .jpg for dashboard screenshots), Compressed archives (.zip)

Deliverable 4: Performance and Security Report

→ **Question Prompt:** What performance metrics and security improvements demonstrate your system's production readiness, and how do your results (load testing, latency percentiles, 60% cost reduction, vulnerability fixes) prove the system meets GlobalSupport Solutions' enterprise requirements that drove Project Phoenix?

→ **Response Type:** File Upload

→ **File Types:** Text (.txt, .pdf, .docx, .md), Image (.png, .jpg for charts), Video (.mp4, .mov - max 15 min for demo)

Deliverable 5: Professional Implementation Reflection

- **Question Prompt:** " How did you synthesize all course learnings (modular components, systematic refactoring, production patterns) to build this production-grade customer support assistant, what critical trade-offs did you balance, and what key insights would you apply differently in future enterprise LLM projects? (400-500 words) "
- **Response Type:** Rich Text or File Upload
- **File Types:** Text (.txt, .pdf, .docx, .md), Audio (.mp3, .wav for verbal reflection), Video (.mp4, .mov - max 15 min)

Instructions for Submission

File Type Selection Guide

Always offer multiple formats for inclusivity:

Standard Options:

- **Text:** .txt, .text, .csv
- **Image:** .png, .jpg, .jpeg, .jfif, .webp, .bmp, .tiff, .tif
- **Video:** .mp4, .mov, .webm (15 min max)
- **Audio:** .mp3, .wav, .aac, .flac
- **Coding:** .bash, .bat, .c, .cpp, .css, .go, .htm, .html, .java, .json, .jsp, .jsx, .md, .php, .py, .r, .scss, .sh, .sml, .sql, .swift, .ts, .tsx, .xml, .yaml, .js

Size Limits:

- Images: 5 MB maximum
- Files: 2 GB maximum
- Videos: 15 minutes maximum

Share The Project

Document: Save the report in PDF format.

Upload: Share in the “Peer Review” area of the course shell with a brief description.

Instructions for Documenting and Sharing The Project for Peer Review

1. Document the Project

- Use word processing software to create your report.
- Ensure all sections of the project document structure are completed.
- Proofread and edit your report for clarity and accuracy.

2. Share the Project

- Save your report in PDF format.
- Upload your documents to the “Peer Review” area in the course shell.
- Provide a brief description of your project in the submission post.

Instructions for Documenting and Sharing The Project for Peer Review

3. Peer Review

- Review the projects submitted by your peers.
- Provide constructive feedback and comments on their work.